

《如何成为 AI 开发工程师》课堂笔记

第一篇 AI 时代的软件工程变革

第 1 章 软件工程进入 3.0 时代

1.1 变革的三大驱动力

软件工程正在被以大模型为代表的人工智能技术深度重塑，形成智能软件工程的新规范。

三个维度展开：

- 技术发展的必然性**——大模型的成熟，使其具备了参与软件工程的核心能力，从代码补全走向跨文件协作开发。
- 产业变革的驱动**——软件行业需要新的方式来应对日益复杂的需求和激烈的市场竞争，传统研发模式效率瓶颈凸显。
- 拥抱 AI 与重塑生产关系**——人机协同将改变软件开发的生產关系与人才结构，开发者角色从“写代码”转向“下指令、把质量”。

1.2 从试点到全面智能化的实施路线图

企业级 AI 落地的推进路线通常分为四个阶段，每一阶段都有明确的能力沉淀重点：

阶段	重点动作	能力沉淀
现状分析	梳理业务痛点、评估现有研发流程	识别 AI 可赋能的高价值场景
能力建设	提示工程、RAG、Agent、数据治理、模型工程	核心技术栈与人才储备
流程重塑	规范先行，将 SDD、Skill 嵌入研发流水线	可复用的工程规范
全面应用	人机协同覆盖需求→设计→编码→测试→部署	端到端智能化交付能力

1.3 本章小结

AI 已经具备了端到端参与软件工程的能力，但真正成功的实施需要构建好提示工程、智能体编排与流程规范三大支柱，并把每一个能力点沉淀到团队可复用的工程资产中。

第二篇 AI 开发工程师岗位认知

第 2 章 传统开发与智能开发的本质差异

2.1 职责边界的迁移

软件工程 3.0 时代下，需求、设计、编码、测试等岗位职责的边界正变得越来越模糊，每个人都可能成为覆盖端到端流程的“超级个体”。

- 传统开发：**需求 → 设计 → 编码 → 测试 → 部署由不同角色分工完成，高度依赖个人经验与手工协作。
- 智能开发：**单一角色即可借助 AI 智能体串联需求分析、智能设计、自动部署等环节。
- 角色重定义：**开发者要从“写代码的人”升级为“指挥 AI 的人”，强调需求洞察、模块化设计与质量把控。

2.2 传统开发 vs. 智能开发对比

维度	传统开发	智能开发
流程串联	依赖多人协作，环节独立	AI 深度融合，一人多角色
瓶颈环节	人工经验、协作沟通	提示词质量、规范定义
交付周期	需求变更易延期	需求清晰即可快速迭代
质量保障	依赖测试与评审	规范先行 + AI 自验证
人才结构	分工明确、专业门槛高	超级个体 + 协作平台

第 3 章 岗位使命与核心职责

3.1 三个维度的工作定位

- 岗位使命：**利用 SDD、AI 工具完成模块的原生设计与开发，参与智能体/AI 应用的设计与平台部署。
- 核心职责：**依据 SDD 规范完成开发、推动 AI 工具落地、辅助单元测试、保障代码质量与交付效率。

3. **可持续能力**：支撑项目长效运转，把 AI 工具沉淀为团队资产，主动输出版本与文档。

3.2 AI 开发特色能力

- 1. **驾驭 AI 工具**：将 Claude Code、open code、Codex 等纳入日常工作流。
- 2. **极速代码框架生成**：用 AI 在数十分钟内完成 MVP 原型搭建（如类抖音短视频平台）。
- 3. **错误分析与调试**：能阅读 AI 输出、定位问题、给出修改指令，让 AI 去修复。
- 4. **工作留痕与文档**：把 AI 协助生成的过程材料归档，形成可追溯的项目履历。

第 4 章 专业技能与岗位能力模型

4.1 硬技能基础

- 1. **编程基础**：至少熟练掌握一门主语言，举一反三地理解其他语言的工程结构。
- 2. **工程开发能力**：掌握需求分析、设计、编码、维护的方法论并选择合适框架。
- 3. **AI 工具应用**：熟练使用主流 AI 开发助手，理解其能力边界与最佳实践。
- 4. **AI 基本原理**：了解模型推理、提示词与大模型接口报文结构，便于系统集成。

4.2 软硬协同能力矩阵

能力维度	关键问题	进阶路径
需求洞察	能否把模糊需求拆解为可验证假设？	结构化拆解 + 提示词工程
模块化设计	能否把复杂系统拆为独立模块？	DDD 思想 + 边界定义
规范与编码	能否定义清晰规范并让 AI 遵循？	SDD 规范先行
单元/接口测试	能否让 AI 自动跑通接口用例？	接口契约 + AI 自测
文档与留痕	能否把每日工作沉淀为知识？	模板化工作日志

第三篇 AI 开发核心概念与方法论

第 5 章 SDD：规范驱动开发

5.1 从 Vibe Coding 到 SDD

Vibe Coding（氛围编程）让 AI 直接上手工作，产出效果不可控；SDD（Spec-Driven Development，规范驱动开发）则要求先固定规范与标准，再让 AI 执行。

- 1. **定义清晰：**业务目标、技术选型、接口契约、验收标准全部预先固化。
- 2. **规范先行：**先把规范定义清楚，杜绝需求模糊、口头约定，让 AI 不再“自由发挥”。
- 3. **规格为元：**所有代码、接口、用例都以规格为依据，需求变更先修订规格。

SDD 的本质是把“意图与实现分离”：人负责定义业务和约束，AI 负责实现，统一团队认知、减少需求偏差，并天然适配 AI 辅助开发。

5.2 SDD vs. Vibe Coding

对比项	Vibe Coding	SDD
工作方式	AI 直接动手实现	规范先行 + AI 执行
产出可控性	效果不稳定、容易走偏	产出可预期，便于验收
需求变更	重新组织语言让 AI 重做	先修订规格，再让 AI 执行
适用场景	原型试错、个人项目	企业级、团队协作项目
质量保障	完全依赖运气与反复试错	基于规格自动测试与对齐

第四篇 AI 开发工具与端到端实操

第 6 章 MCP 协议与 Skill 技能包

6.1 MCP：模型上下文协议

MCP（Model Context Protocol）是 Anthropic 提出的一项开放协议，让大模型能以标准化方式安全地调用外部工具，被称作“AI 的 USB-C 接口”。

- 1. **标准化协议：**不同模型、不同工具按统一规范对接，避免重复定制 function call。
- 2. **工具发现：**模型可发现并理解 MCP Server 暴露的接口、参数与报文结构。
- 3. **跨平台安全：**协议本身明确权限与边界，便于企业级数据安全与权限控制。
- 4. **生态丰富：**天气、数据库（SQLite/Redis 等）、联网、文档生成等场景都已有现成 Server。

6.2 Skill：技能包

Skill 即技能包，用于把企业专属能力或个性化流程“教会”通用大模型，让模型在特定任务上稳定地产出高质量结果。

- 1. **拓展 AI 边界：**把思考与执行串联起来，让通用模型承担特定专业任务。
- 2. **积木式编排：**把多个 Skill 组合即可完成复杂任务，能在 openspec、open code 等智能体里直接挂载使用。
- 3. **低成本迭代：**相比训练专用模型，Skill 几乎无成本即可完成能力扩展与升级。

第 7 章 常用工具：gsgo 与 openspec 框架

7.1 企业内部工具 gsgo

gsgo 是公司基于 open code 二次封装的企业级 AI 开发助手，覆盖办公辅助、项目文档、业务创新、编码、技术排查等场景。

- 1. **项目文档编写：**自动生成方案、接口、变更说明等结构化文档。
- 2. **代码生成与重构：**理解整个代码结构后给出方案、生成图片/文字描述与代码。
- 3. **Bug 修复与排障：**通读代码上下文，定位问题并给出修复建议。

7.2 openspec：轻量级 SDD 框架

openspec 是一套支持多种 AI 智能体协同工作的规范驱动开发框架，核心命令包括：explore（探索）、proposal（提案）、apply（实施）、archive（归档）。

- 1. **探索（explore）：**通过多轮对话澄清需求，敲定技术方案、流程与验收口径。
- 2. **提案（proposal）：**自动输出需求规格说明书、任务清单与设计文档。
- 3. **实施（apply）：**按任务清单逐步生成代码并自验证，直到所有任务打勾。
- 4. **归档（archive）：**按日期归档到制品库，便于日后复用与回溯。

环节	关键产物	团队价值
探索	澄清后的需求描述、技术选型、流程	减少需求漂移
提案	规格说明、设计文档、任务清单	统一认知、便于评审
实施	代码、配置、自动化测试	降低编码与维护成本

归档	日期规范的成果包	知识资产沉淀
----	----------	--------

第 8 章 端到端实操流程与课后作业

8.1 类抖音视频播放 Demo 全流程

课程现场演示的短视频播放应用，完整跑通了“探索→提案→实施→验证→归档”五步流程，仅用十余分钟便生成可运行的 MVP。

1. **需求澄清**：首页/上传/我的；竖屏滚动；上下页切换；本地存储；无需登录。
2. **技术选型**：前端 Vue、后端 Spring Boot、本地 SQLite、无状态 JWT。
3. **架构确认**：前后端分离、目录结构、API 设计、数据库表与字段——明确。
4. **任务执行**：AI 智能体按清单生成前台、后台、上传与登录注册等全部功能。
5. **验证发布**：完成注册 → 上传视频 → 首页播放 → 点赞与个人中心全链路打通。

8.2 实操要点与风险

1. **规范必须前置**：清晰的功能、技术与验收定义是避免 AI 失控的前提。
2. **文档必须审阅**：人是机协同中的“导演”，对每个 AI 生成的文档都要看一遍。
3. **留痕必不可少**：把 AI 的修改、测试、归档记录写入项目史，便于回溯与协作。

课后作业：以个人为中心，独立完成一个类抖音的短视频播放 Web 应用：前端 Vue、后端自选语言、SQLite 本地库；至少实现注册登录、上传视频、首页播放、点赞与个人中心五大功能，并按规范归档。

8.3 课程总结

今天的课程从宏观的 AI 时代软件工程变革，落到 AI 开发工程师的岗位定位，再到 SDD/MCP/Skill 等核心概念与 openspec 框架的实操闭环，让我们看清了“人是机协同中的主导者，AI 是被指挥的工具”这一根本定位。

坚持夯实编程基础，主动参与端到端交付，把每一次 AI 协作都沉淀为可复用的工程资产——这是 AI 时代开发工程师的核心竞争力。